

Cifra de Vigenère

Implementação e Criptoanálise

Um cifrador/decifrador construído do zero e um ataque estatístico de recuperação de chave, baseado no índice de coincidência e no teste qui-quadrado.

Rafael de Lima Pereira
Matrícula 242043277

Pedro Freitas França
Matrícula 232006028

Rian Kallebe da Silva Lisboa
Matrícula 242012000

Arthur Martins Pereira de Souza
Matrícula 241004499

01

Introdução

O problema, a intuição histórica e o roteiro do trabalho.



Uma cifra considerada indecifrável por três séculos

Descrita no século XVI, a cifra de Vigenère é uma **substituição polialfabética**: a mesma letra do texto claro pode virar letras diferentes no criptograma, dependendo da posição em que ocorre.

"*le chiffre indéchiffrable*"

é justamente porque o achatamento das frequências de letras confunde a análise de frequência clássica.

O achatamento é apenas parcial: **a chave se repete ciclicamente**. Descoberto o comprimento da chave, o criptograma se decompõe em várias cifras de César — cada uma vulnerável à análise de frequência.

A IDEIA CENTRAL

$$c_i = (p_i + k_i \bmod \ell) \bmod n$$

$$p_i = (c_i - k_i \bmod \ell) \bmod n$$

$\ell = 1$ degenera na cifra de César

ℓ pequeno, m grande faixa de uso prático — a fragilidade

$\ell \geq m$, chave aleatória cifrador de Vernam — seguro

Objetivo e estrutura do trabalho

O trabalho está dividido em duas partes complementares, implementadas em Python 3 sem qualquer biblioteca de criptografia pronta.

PARTE I

Cifrador / Decifrador

Implementação das equações de cifração e decifração, com tratamento cuidadoso de acentos, caixa e pontuação (vigenere/cipher.py).

PARTE II

Criptoanálise

Ataque de recuperação de chave sobre dois criptogramas de idioma conhecido (PT e EN) mas chave desconhecida, via IC e qui-quadrado (vigenere/attack.py).

CÓDIGO-FONTE COMPLETO

github.com/RafaelLime/vigenere

Sem bibliotecas de criptografia — apenas unicodedata, collections, string, argparse, dataclasses.

02

Parte Técnica

Decisões de implementação da cifra e metodologia do ataque estatístico.



Núcleo da cifra

Um único laço percorre a mensagem caractere a caractere, mantendo um índice próprio para a chave. O parâmetro `sign` vale `+1` na cifração e `-1` na decifração — uma única rotina implementa as duas equações.

- `key_index` avança apenas quando um caractere é efetivamente cifrado → a chave nunca perde a sincronia.
- Núcleo matemático isolado em `_transform` (`vigenere/cipher.py`), reaproveitado pela cifração, decifração e pelo ataque.
- Suíte de 26 testes automatizados (`pytest`), incluindo o vetor clássico `ATTACKATDAWN / LEMON` → `LXFOPVEFRNHR`.



`cipher.py`

```
for char in text:
    letter = fold(char)
    if letter is None:
        if alphabet == "preserve":
            result.append(char)
        continue

    base = ord("A") if letter.isupper() \
           else ord("a")
    shift = offsets[key_index %
                    len(offsets)] * sign
    shifted = chr(
        (ord(letter) - base + shift)
        % ALPHABET_SIZE + base
    )
    result.append(shifted)
    key_index += 1
```

Tratamento do alfabeto e dos acentos

Entrada	Tratamento
a-z / A-Z	Cifradas. Caixa preservada em preserve; forçada a maiúscula em strict.
Acentuadas (á ã ç é ô ü ...)	Reduzidas à letra base por decomposição Unicode NFD, depois cifradas. O acento não é restaurado ao decifrar.
Espaços, dígitos, pontuação	preserve: copiados sem consumir letra da chave. strict: descartados.
Caracteres da chave	Reduzidos da mesma forma; o que não é letra é ignorado. Chave sem nenhuma letra é erro.

Por quê reduzir acentos? Preservá-los vazaria o idioma do texto original no criptograma. A normalização NFD decompõe ç em c + cedilha combinante, tratando qualquer letra latina acentuada — não apenas uma lista fixa.

ação → acao
perda ortográfica, não semântica

Metodologia do ataque: duas etapas

A cifra é a intercalação de ℓ cifras de César. O ataque explora exatamente isso, em duas etapas independentes.

1

Estimar o comprimento da chave

Índice de Coincidência (IC)

$$IC = \sum n_i (n_i - 1) / N(N-1)$$

Testa-se $k = 1 \dots 20$. Para cada k , o texto é dividido em k colunas e calcula-se o IC médio.

Se k é o comprimento correto (ou múltiplo), cada coluna é uma cifra de César pura $\rightarrow$ IC salta para $\approx 0,07$ (idioma natural).

Se k está errado, deslocamentos se misturam $\rightarrow$ IC permanece perto de 0,038 (uniforme).

Escolhe-se o k que maximiza o IC médio.

2

Recuperar cada letra da chave

Teste qui-quadrado (χ^2)

$$\chi^2 = \sum (O_i - E_i)^2 / E_i$$

Fixado k , cada uma das k colunas é um César independente.

Testam-se os 26 deslocamentos possíveis; a distribuição resultante é comparada às frequências do idioma (Tabela de frequências PT/EN).

O deslocamento de menor χ^2 é a letra candidata daquela posição da chave.

Concatenar as letras vencedoras reconstrói a chave.

Arquitetura do projeto (github.com/RafaelLime/vigenere)

<code>cipher.py</code>	Parte I	Cifração e decifração; núcleo matemático em <code>_transform</code> .
<code>cli.py</code>	Parte I	Interface de linha de comando (encode / decode).
<code>attack.py</code>	Parte II	IC, análise de frequência, separação em colunas, χ^2 e reconstrução da chave; tabelas PT/EN.
<code>integration.py</code>	Parte II	Orquestra o ataque (testa idiomas/comprimentos), avalia e refina o resultado; menu interativo.
<code>tests/</code>	Ambas	Suíte pytest cobrindo cifra, CLI, primitivas estatísticas e o ataque de ponta a ponta.

Nenhuma biblioteca pronta de criptografia ou criptoanálise é usada — apenas módulos padrão: `unicodedata`, `collections`, `string`, `argparse`, `dataclasses`.

03

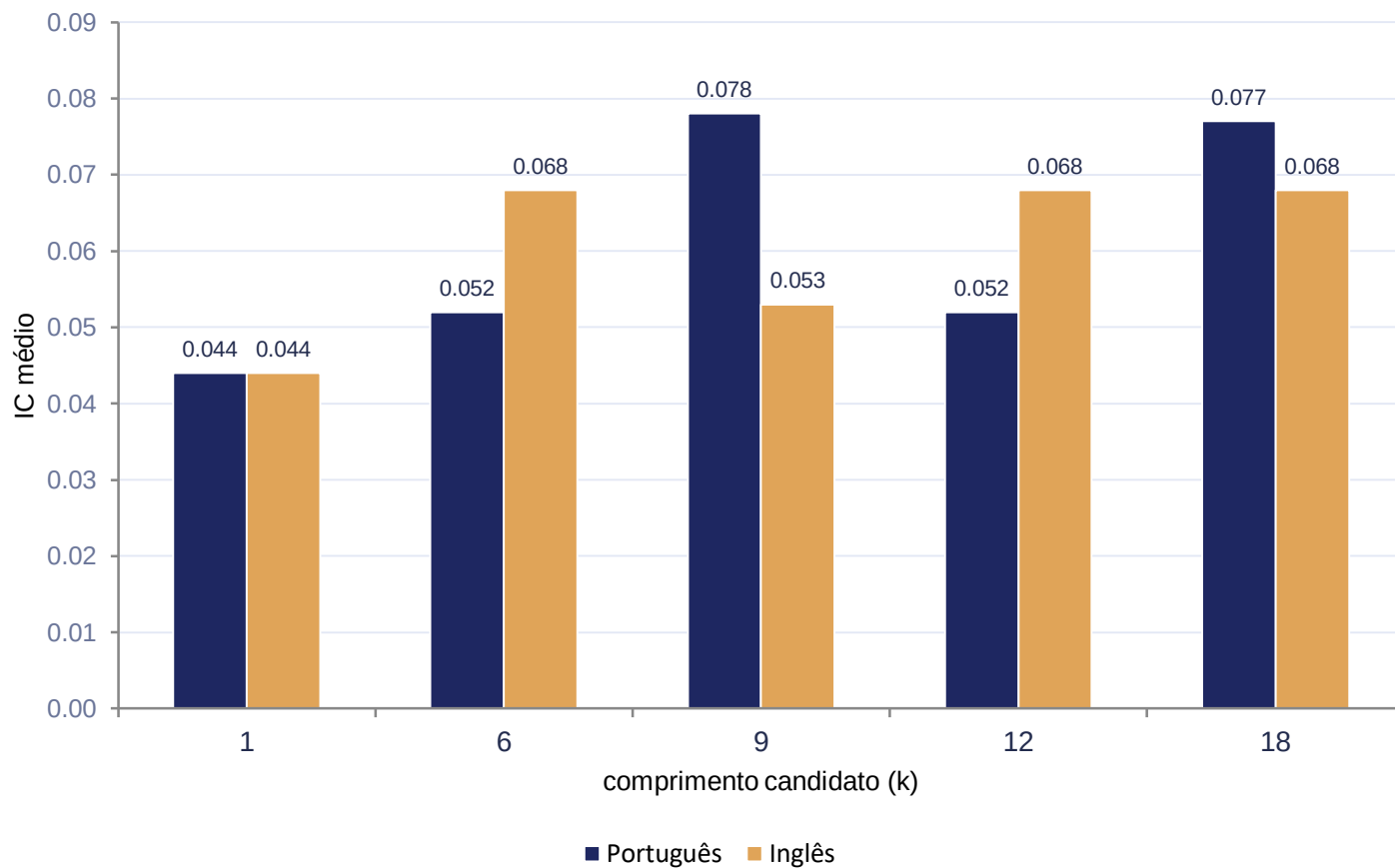
Resultados & Demo

O ataque aplicado aos criptogramas de validação em português e inglês.



Etapa 1 — o IC médio revela o comprimento da chave

IC médio das colunas por comprimento testado (k)



k SELECIONADO

Português

k = 9

IC médio = 0,078 · 762 letras

Inglês

k = 6

IC médio = 0,068 · 697 letras

Picos secundários em múltiplos exatos (18 e 12) confirmam a periodicidade da chave.

Etapa 2 — χ^2 recupera cada letra com folga ampla

Candidatos de χ^2 por posição (menor é melhor)

Pos.	PT (k=9)	EN (k=6)
0	S: 22,1 vs 749 / 758	C: 25,7 vs 374 / 386
1	E: 14,0 vs 813 / 919	I: 20,6 vs 542 / 579
2	G: 21,7 vs 204 / 398	P: 32,0 vs 422 / 473
3	U: 17,5 vs 514 / 694	H: 49,8 vs 237 / 250

Em nenhuma posição foi preciso recorrer ao 2º ou 3º colocado.

CHAVES RECUPERADAS

Português

SEGURANCA

IC do texto claro: 0,079 (esperado $\approx$ 0,074)

Inglês

CIPHER

IC do texto claro: 0,067 (esperado $\approx$ 0,066)

Ambos os ataques foram totalmente automáticos — nenhum ajuste manual foi necessário.

A cifra e o ataque em ação, via linha de comando

A implementação expõe um comando `vigener`, e o ataque completo pode ser chamado tanto pelo menu interativo (`python -m vigener.integration`) quanto como biblioteca, via `resolver_criptograma(...)`.

```
terminal - vigener CLI

$ vigener encode "ATTACKATDAWN" --key LEMON
LXFOPVEFRNHR

$ vigener decode "LXFOPVEFRNHR" --key LEMON
ATTACKATDAWN

$ vigener encode "Ataque ao amanhecer!" --key segredo --alphabet strict
SXGHYHOGESRRKSUIX

$ python -m vigener.integration
=== Cifra de Vigenère ===
Selecione o idioma do texto:  1. Português  2. Inglês
> 1
Selecione a operação:  3. Atacar um criptograma (chave desconhecida)
> 3
Digite o criptograma: SGXZTWCYVGWMDSSTXRXLQSIU...

-> Etapa 2: k=9 selecionado (IC médio 0.078)
-> Etapa 3: chave recuperada = SEGURANCA
-> Resultado confiável (IC do texto claro: 0.079)
```

04

Conclusão

Volume de texto

IC e χ^2 convergem lentamente; colunas curtas (poucas dezenas de letras) tornam a estatística ruidosa.

Dependência do idioma

Pressupõe frequências conhecidas do idioma alvo; textos técnicos ou mistos degradam o ataque.

Múltiplos do comprimento

O IC não distingue k de seus múltiplos — exige inspeção da legibilidade do texto decifrado.

Escopo do alfabeto

Apenas 26 letras são cifradas; dígitos e pontuação são preservados ou descartados, não protegidos.

A repetição periódica da chave é o ponto de ruptura

Ela permite decompor o problema em subcifras de César independentes: a segurança da cifra depende da **razão entre o comprimento da chave e o do texto** — não do tamanho do espaço de chaves.

OBRIGADO

Código-fonte, testes e este relatório completo:

`github.com/RafaelLime/vigenere`

Rafael de Lima Pereira · Pedro Freitas França · Rian Kallebe da Silva Lisboa · Arthur Martins Pereira de Souza